

Báo cáo cuối kỳ: SCRFD, SPOS-NAS và ASR+JSAR

23127333 Trương Quốc Cường 23127011 Lê Anh Duy 23127199 Trần Gia Huy

CSC14006 – Nhận dạng
Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM

TP. Hồ Chí Minh, ngày 20 tháng 4 năm 2026



- 1 Mở đầu
- 2 SCRFD Gốc
- 3 Reproduction
- 4 SPOS-NAS Cho CR
- 5 ASR+JSAR: Điều chỉnh tỉ lệ cắt và gán mẫu khi huấn luyện
- 6 Kết luận
- 7 Tài liệu tham khảo

Mở đầu

Phạm vi của báo cáo này

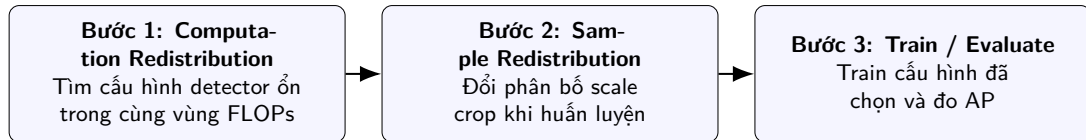
- Báo cáo chỉ tập trung vào bài báo **SCRFD** và pipeline detector một giai đoạn cho face detection trên **WIDER FACE**.
- Nội dung được tổ chức theo đúng logic của một report bài báo khoa học: trình bày phương pháp, tái hiện bằng official code, đối chiếu paper với code, rồi phân tích kết quả.
- Từ đó nhóm mở rộng theo đúng hai mắt xích quan trọng của SCRFD:

Hai hướng mở rộng

SPOS-NAS cho bước **Computation Redistribution**;

ASR+JSAR cho **việc điều chỉnh tỉ lệ cắt và gán mẫu** trong lúc huấn luyện.

SCRFD nên được nhìn như pipeline 3 bước



- Điểm quan trọng là ba bước này gắn chặt với nhau: kiến trúc đúng nhưng phân phối tỉ lệ cắt khi tăng cường dữ liệu sai vẫn tụt Hard AP.
- Vì vậy phần mở rộng cũng phải đi theo đúng trình tự **CR trước, SR sau**.

SCRFD Gốc

- SCRFD hướng vào **face detection hiệu quả**, đặc biệt là nhóm **tiny faces** trong ảnh VGA.
- Trên WIDER FACE, split Hard chứa rất nhiều mặt cực nhỏ, nên detector không thể chỉ mạnh về ngữ nghĩa sâu; nó còn phải giữ tốt chi tiết không gian ở các tầng nông.
- Bài báo vì thế không chỉ hỏi mô hình nào mạnh hơn, mà hỏi hai câu đúng hơn:

Hai câu hỏi cốt lõi

Với cùng ngân sách FLOPs, compute nên đặt ở đâu?

Và trong lúc train, detector nên nhìn thấy khuôn mặt ở những scale nào nhiều hơn?

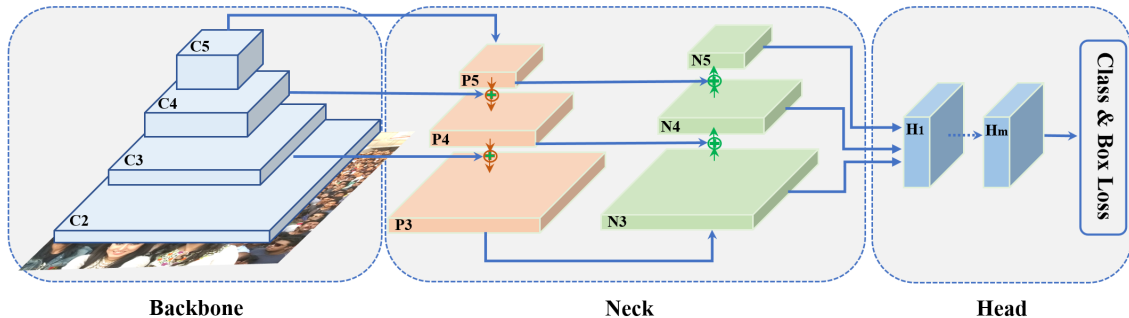
Minh họa WIDER FACE Hard và tiny faces



Kiến trúc nền Backbone–Neck–Head

- **Backbone:** tạo chuỗi đặc trưng $C_2 \rightarrow C_5$.
- **PAFPN:** hợp nhất thông tin nhiều mức để vừa giữ vị trí vừa giữ ngữ nghĩa.
- **Dense head:** dự đoán classification và bbox trên các feature map.
- Trọng tâm của SCRFD là làm cho cấu trúc này phù hợp hơn với tiny-face detection thay vì bê nguyên detector phân loại ảnh sang.

Minh họa kiến trúc Backbone–Neck–Head



Computation Redistribution được làm theo hai pha

- **Pha 1: search backbone.** Mục tiêu là cô lập câu hỏi riêng cho backbone: trong khung 4 stage C_2, C_3, C_4, C_5 , compute nên dồn nhiều hơn vào stage nào.
- Ở repo public demo, bước này mặc định sinh **64 candidate backbone**; paper mô tả quy mô đầy đủ là **320 candidate**.
- Mỗi candidate đều bị lọc lại theo cùng vùng chi phí, trong code là khoảng **target FLOPs $\pm 2\%$** , nên chỉ những cấu hình còn quanh 2.5G mới được giữ lại để train và so sánh.
- Các cấu hình hợp lệ sau đó được **train rút gọn 80 epoch**, đánh giá bằng **Hard mAP**, rồi chọn **config tốt nhất** làm template cho pha 2.
- **Không có bước lấy top 10% hay khoảng tin cậy thống kê**; đây là ranking-based search: sinh candidate $\rightarrow$ lọc FLOPs $\rightarrow$ train ngắn $\rightarrow$ chọn config thắng.

Các bậc tự do mà CR thực sự đi tìm

- **Pha 1 – search backbone:** không search từ số 0, mà đi trong một khung 4 stage cố định rồi thay đổi các bậc tự do chính của backbone.
- Các bậc tự do ở bước này là: **loại block, base width ban đầu, độ sâu từng stage, và tỉ lệ độ rộng giữa các stage.**
- Nói ngắn gọn, pha 1 đang đi tìm **hình dạng backbone:** stage nào sâu hơn, stage nào rộng hơn, và độ dốc tăng width giữa $C_2 \rightarrow C_5$ nên như thế nào.
- **Pha 2 – search toàn mạng:** từ backbone template thắng ở pha 1, paper không mở toàn bộ detector một cách tự do, mà chỉ cho backbone dao động quanh template đó rồi tinh chỉnh tiếp **neck và head.**
- Các bậc tự do ở bước này là: **mức scale lại độ sâu backbone, mức scale lại độ rộng backbone, số kênh của neck, số kênh của head, và độ sâu của head.**
- Ý nghĩa của cách tách này là tránh để neck/head che mờ tín hiệu thật sự của backbone ở đầu quy trình, rồi chỉ sau đó mới phân bổ nốt compute cho toàn detector.

Quy trình tìm kiếm kiến trúc CR



Giả định và giới hạn của CR

- CR không search từ số 0. Paper giả định trước một **khung detector cố định**: backbone 4 stage, PAFPN, dense head, các mức stride chính vẫn giữ nguyên.
- Trong pha backbone, search thực chất là phân bổ lại depth/width quanh template ban đầu, chứ không mở tự do mọi loại module.
- Trong pha toàn mạng, detector cũng không được mở hoàn toàn; backbone đã bị khóa quanh template thẳng, rồi mới phân phối nốt compute cho neck và head.
- Vì vậy giá trị của CR không phải ở chỗ paper search mọi thứ, mà ở chỗ paper chọn đúng abstraction level để search: **phân bổ compute giữa các phần của detector**.

Sample Redistribution tác động vào đầu

- ➊ Mỗi ảnh train được crop vuông theo một scale rồi resize về cùng input 640×640 .
- ➋ Vì vậy cùng một khuôn mặt sau resize có thể lớn hơn hoặc nhỏ hơn tương đối, dù ảnh đầu vào là cùng một ảnh.
- ➌ Kích thước tương đối đó quyết định mặt rơi vào feature level nào và có bao nhiêu positive anchors.
- ➍ Do đó SR không chỉ là augmentation; nó đang điều khiển trực tiếp **training signal** cho tiny/small faces.

Luồng nhân quả

bộ tỉ lệ cắt $\rightarrow$ face size sau resize $\rightarrow$ positive samples $\rightarrow$ Hard AP

Giả định và giới hạn của SR

- SR cũng không tối ưu trực tiếp mật độ mẫu dương; paper chỉ search trên một **tập tỉ lệ cắt rời rạc** rồi để pipeline crop/resize biến nó thành tín hiệu học.
- Nghĩa là paper tối ưu một biến gián tiếp: bộ tỉ lệ cắt, chứ không tối ưu trực tiếp số mẫu dương hay gradient mass theo level.
- Public release còn cho thấy narrative của paper và code public không hoàn toàn trùng nhau: code công khai giữ một danh sách tỉ lệ cắt dựng tay, trong khi paper nhấn mạnh bộ tỉ lệ cắt được tìm ra.
- Điểm này rất quan trọng khi đọc kết quả reproduction: chênh lệch Hard AP có thể đến từ chính phân phối tỉ lệ cắt khi tăng cường dữ liệu, không chỉ từ backbone hay random seed.

Điểm mạnh và hạn chế của SCRFD

Điểm mạnh

- Đặt đúng bài toán tiny-face detection dưới ràng buộc hiệu quả.
- Nối được kiến trúc và dữ liệu train thành một pipeline logic.
- Đường cong accuracy-efficiency rất thực dụng cho face detection.

Hạn chế

- Search vẫn tốn tài nguyên.
- Search space còn bị ràng buộc bởi template detector ban đầu.
- Kết quả gắn rất mạnh với WIDER FACE và tiny-face benchmark.

Reproduction

WIDER FACE là benchmark trung tâm của báo cáo

Split	Số ảnh	Số face boxes	Ghi chú
Train	12,880	159,393	dùng để train
Val	3,226	39,697	dùng để evaluate
Test	16,097	–	không có GT public

- Paper thực chất bám rất chặt vào WIDER FACE để search, train và benchmark.
- EDA trong report cho thấy hơn 70% face boxes nằm dưới ngưỡng 32 px, nên tiny faces không phải edge case mà là vùng dữ liệu thống trị bài toán.

Cấu hình baseline chạy lại paper

- Nhóm bám **official public SCRFD codebase** và official detector family, không tự viết lại mô hình.
- Baseline được chọn là **SCRFD-2.5GF**: đủ lớn để có ý nghĩa thống kê, nhưng vẫn nằm trong phạm vi tài nguyên có thể tái hiện ổn định.
- Đây là quyết định có chủ ý: full NAS của paper quá nặng vì phải sinh nhiều candidate, train rút gọn, đánh giá, chọn cấu hình rồi mới train dài cho model cuối.
- Bộ 12 mức tỉ lệ cắt theo bài báo được nhóm **phục dựng lại** trên pipeline công khai; nó không phải một baseline upstream có sẵn đúng nguyên xi từ authors.
- Vì vậy reproduction của nhóm là **public reproducible baseline trên official stack**, không phải rerun toàn bộ NAS end-to-end của paper.

Protocol train và evaluate của baseline

Thành phần	Thiết lập
Input	640×640
Optimizer	SGD, momentum 0.9, weight decay 5×10^{-4}
Schedule	80 epochs, cosine LR, warmup
Augmentation	crop vuông, resize, flip, photometric distortion, normalize
Evaluation	WIDER FACE Easy/Medium/Hard trên validation split , threshold 0.02, NMS 0.45

- Điểm cần nhớ là baseline dùng cùng pipeline train/eval với official repo, nhưng đường chạy '80e cosine + bộ 12 mức theo bài báo' là cấu hình thực nghiệm nhóm dựng lại trên official stack.
- Vì vậy khi so với official model zoo, phải tách rõ hai lớp nguyên nhân: khác recipe train dài-ngắn và khác phân phối tỉ lệ cắt khi tăng cường dữ liệu.

Kết quả tái hiện baseline và vai trò của bộ 12 mức theo bài báo

Thiết lập	Easy	Medium	Hard	mAP
Official model zoo	93.78	92.16	77.87	87.94
Baseline (bộ mặc định)	91.40	89.69	72.49	84.53
Baseline (12 mức theo bài báo)	91.61	89.99	73.71	85.10

- Trong cùng khung **80 epoch cosine**, chỉ riêng việc đổi sang **bộ 12 mức theo bài báo** đã kéo Hard AP tăng +1.22.
- Điều này xác nhận một insight lớn của report: phần khó nhất của SCRFD không chỉ là chạy model, mà là tái tạo đúng phân phối tỉ lệ cắt cho tiny faces.

Đọc đúng gap với official model zoo

- Gap với official không nên quy hết cho một lý do duy nhất.
- **Official 2.5GF** là mốc model zoo đã được huấn luyện rất dài trong public recipe gốc, khoảng **640 epoch với step schedule**.
- **Baseline của nhóm là 80 epoch với cosine scheduler** trên official public codebase, rồi gắn thêm bộ 12 mức theo bài báo vào pipeline công khai.
- Vì vậy gap Hard -4.16 phải được đọc như tổng hợp của:

Bốn nguồn chênh lệch

training horizon, scale policy, randomness, và mức độ tinh chỉnh sâu hơn của pipeline official

SPOS-NAS Cho CR

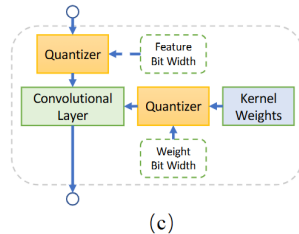
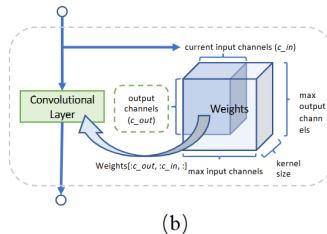
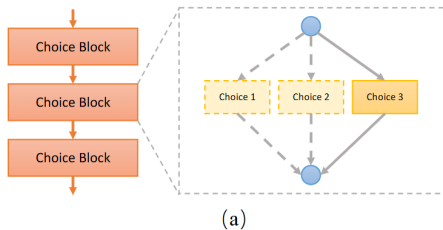
Vì sao dùng SPOS-NAS để mở rộng CR

- Official CR search rất nặng, nên nhóm dùng **SPOS-NAS proxy** để khảo sát sâu hơn cấu trúc phân bổ compute trong cùng miền 2.5 GFLOPs.
- Mục tiêu không phải thay official AP benchmark, mà là kiểm tra xem khi search dưới áp lực tiny-face detection, winner có tự học một quy luật phân bổ compute nhất quán hay không.
- Đây là **phân tích bổ sung cho CR**, không phải nhánh evidence chính mạnh hơn baseline + ASR.
- Đây là phần mở rộng trực tiếp cho **Computation Redistribution**, nên được trình bày trước.

Flow triển khai SPOS-NAS proxy

- 1 Xây dựng một supernet bao phủ nhiều phương án depth/width trên bốn stage $C_2 \rightarrow C_5$.
- 2 Huấn luyện supernet theo kiểu single-path để mỗi mini-batch chỉ kích hoạt một đường con.
- 3 Sau khi supernet đủ ổn định, lấy mẫu **1,000** candidate trong cùng vùng 2.5 GFLOPs.
- 4 Với mỗi candidate, chạy BatchNorm recalibration rồi đo proxy score trên validation.
- 5 Phân tích cấu hình Top-10 / thua để rút ra quy luật.

Minh họa luồng làm việc của SPOS

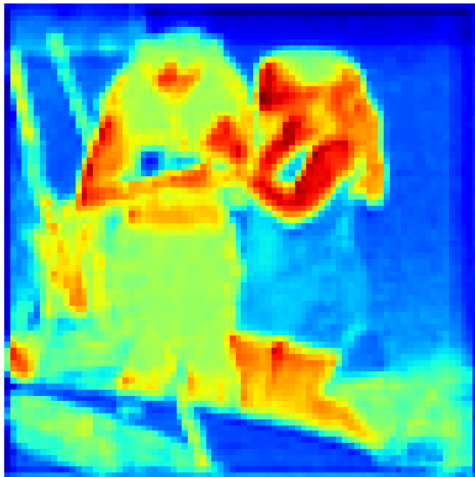


Inference heatmap của kiến trúc tốt nhất

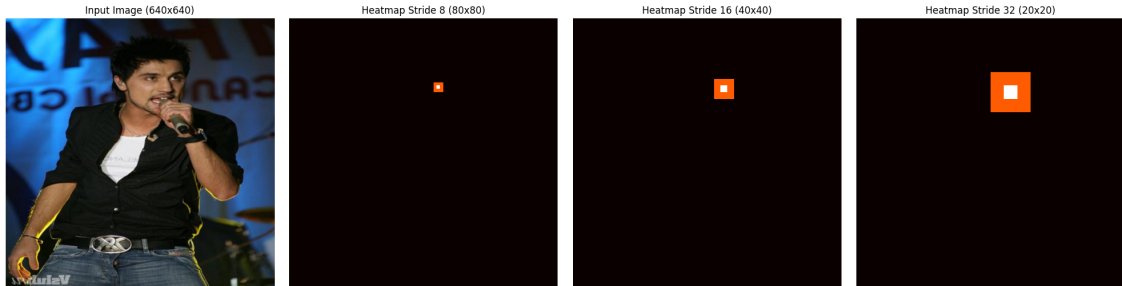
Ảnh gốc



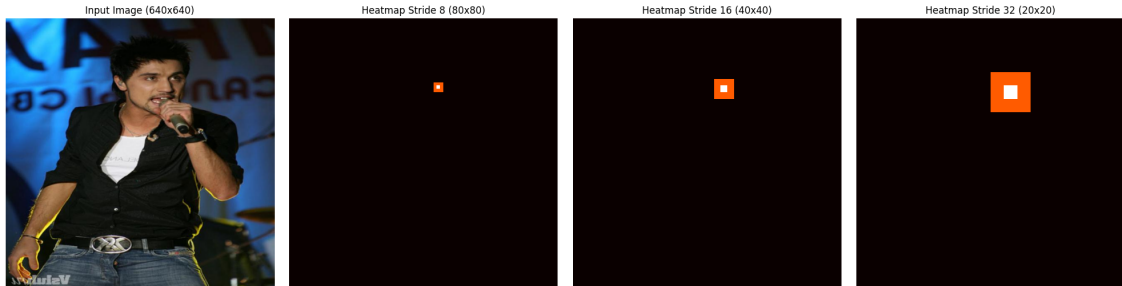
Bản đồ nhiệt dự đoán (Heatmap)



Ví dụ heatmap generated theo edge-focus



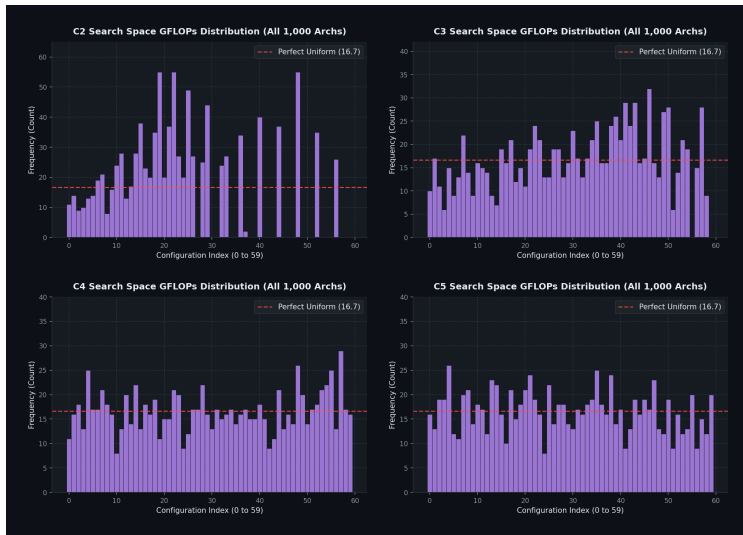
Ví dụ heatmap ngược lại theo center-focus



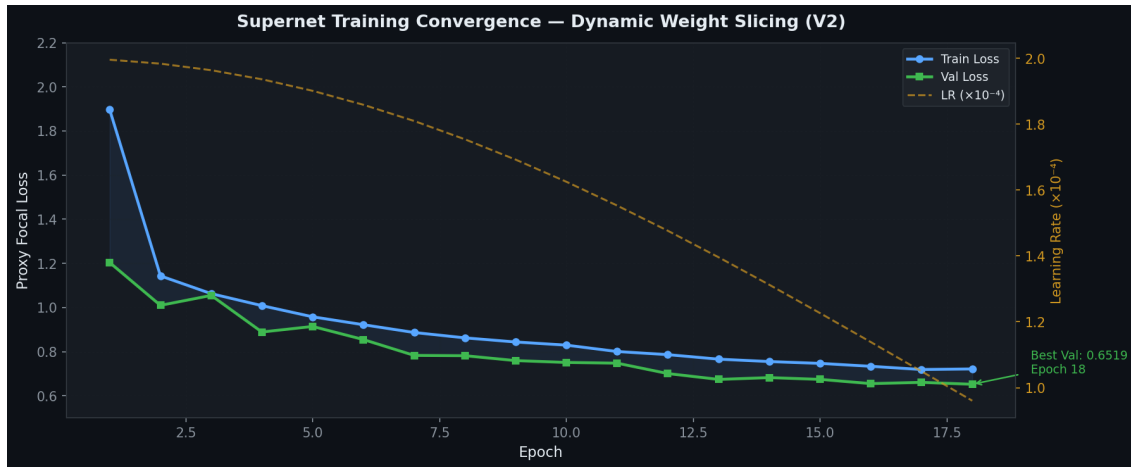
Đánh giá Supernet và Proxy NAS

- Cả supernet center-focus và edge-focus đều hội tụ ổn định sau khi giảm LR.
- Quá trình lấy mẫu sinh ra không gian biến thể bao phủ rất đều đặn, rải rác công bằng cơ hội thay đổi cấu trúc cho mọi tầng.
- Bảng xếp hạng 1,000 candidates cho thấy sự phân hóa rõ rệt dù cùng GFLOPs, tạo cơ sở mạnh để chọn lọc cấu hình tối ưu.

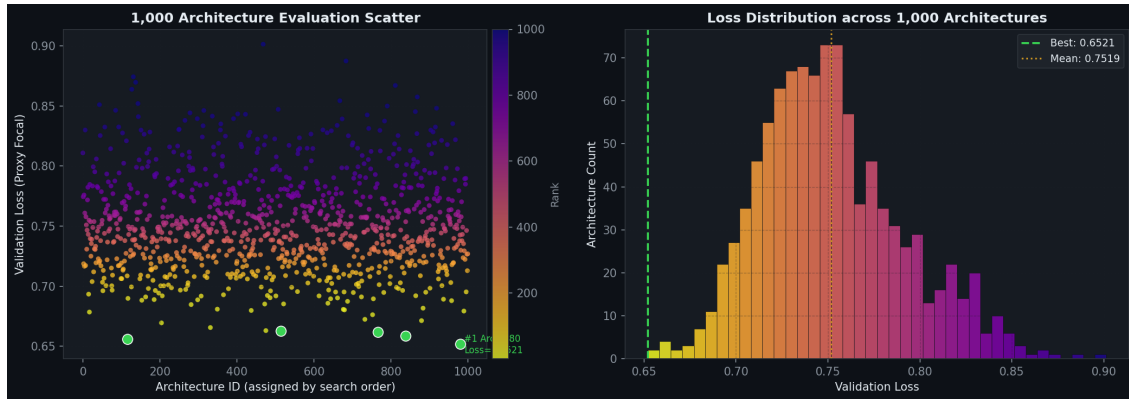
Tính đồng đều của không gian Search (Uniformity)



Đường cong hội tụ của Supernet



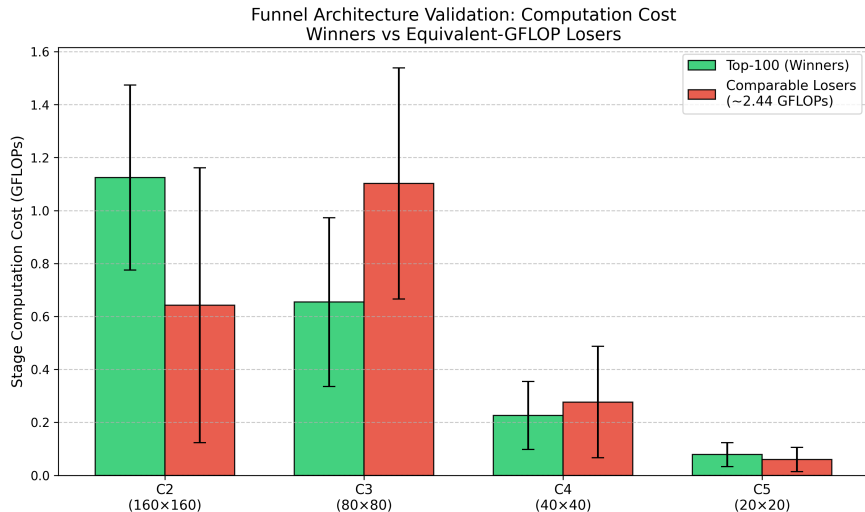
Leaderboard 1,000 Candidate



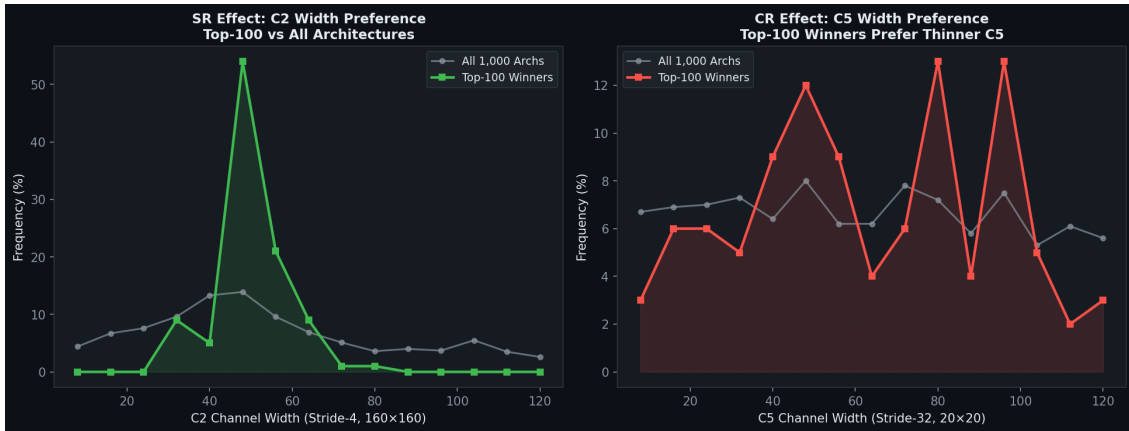
Kết quả chính của SPOS-NAS

- Nhóm tốt có xu hướng **đầu tư mạnh vào** C_2 và tránh làm C_3 phình quá mức.
- C_2 giữ chi tiết vị trí không gian cho tiny faces, còn C_5 bổ sung ngữ nghĩa đủ mạnh cho head.
- C_3 là vùng dễ thành bẫy chi phí: feature map vẫn còn lớn nên chi phí đắt, nhưng gain tăng không tương xứng.
- Khi đổi proxy từ center-focus sang edge-focus, phân bố học dịch chuyển (giảm C_2 tăng C_3) → phản ứng linh hoạt với hình học của bài toán (vì ở viền cần thêm ngữ cảnh không gian nên cần stride lớn hơn).

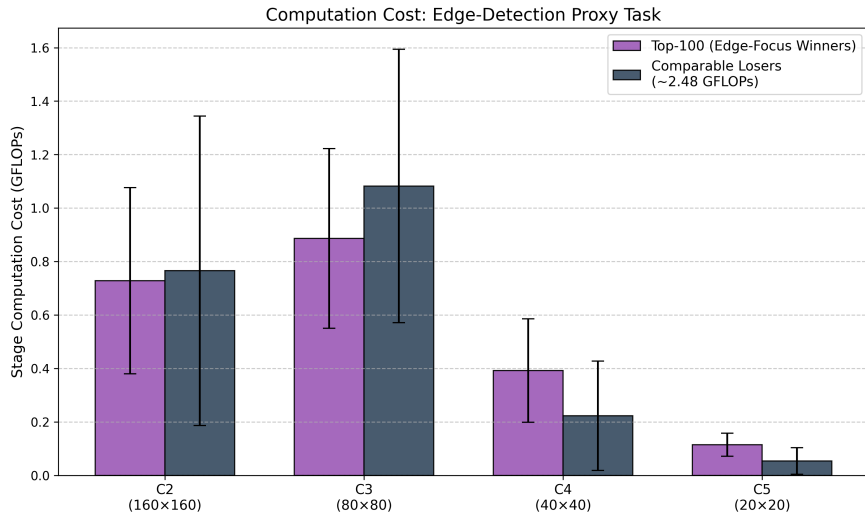
Phân tích cơ cấu phân bố: Tránh bẫy FLOPs ở C3



Top winners dành compute cho C2 và C5



Phân tích cơ cấu phân bố: Tập trung vào biên (Edge-focus)



Điểm mà phần mở rộng này xác nhận

Computation Redistribution của SCRFD là một nguyên tắc phân bổ compute có cấu trúc, không phải mẹo thử-sai ngẫu nhiên.

- Trong miền 2.5 GFLOPs, winner thắng vì phân bổ compute đúng hơn, không phải vì đơn giản lớn hơn.
- Đó là lý do report kết luận SCRFD mạnh vì nó đặt năng lực biểu diễn đúng chỗ ngay từ cấp kiến trúc.

ASR+JSAR: Điều chỉnh tỉ lệ cắt và gán mẫu khi huấn luyện

Động cơ của ASR+JSAR

Adaptive Sample Redistribution (ASR) và **Joint Sample Assignment Redistribution (JSAR)** là hai cải tiến được thêm vào giai đoạn huấn luyện của SCRFD.

- Sau baseline dùng bộ 12 mức theo bài báo, gap lớn nhất vẫn nằm ở split Hard.
- Điều đó cho thấy phân phối chọn tỉ lệ cắt tĩnh vẫn chưa đủ, và tiny faces vẫn còn bị thiếu mẫu dương ở bước gán nhãn.
- Vì vậy nhóm sửa đúng hai nút thắt của giai đoạn train:

Hai cải tiến

Adaptive Sample Redistribution (ASR): điều chỉnh động xác suất chọn các mức tỉ lệ cắt khi tăng cường dữ liệu theo tín hiệu khó dễ đang quan sát được.

Joint Sample Assignment Redistribution (JSAR): tăng số mẫu dương được gán cho tiny faces khi quy tắc gán chuẩn còn bỏ sót.

Nguồn tham khảo: ý tưởng được tham khảo từ **Sample Redistribution** của SCRFD [1] và **ATSS** cho bước gán mẫu [9].

Cấu hình huấn luyện của ASR+JSAR

Thiết lập chung

- Mô hình: **SCRFD-2.5GF**, input 640×640
- Số epoch: **80**
- Phần cứng: **1 × NVIDIA RTX 3090**
- Batch size: **32**
- Optimizer: **SGD**, momentum = 0.9, weight decay = 5×10^{-4}
- Learning rate ban đầu: **0.04**
- Lịch học: **CosineAnnealing**, warmup tuyến tính 1500 iter, warmup ratio = 10^{-3} , min lr = 0

Tham số ASR và JSAR

- ASR: warmup = 2 epoch, update interval = 1000, EMA = 0.8, min prob = 0.03
- Bin kích thước: $[0, 16)$, $[16, 32)$, $[32, 96)$, $[96, +\infty)$
- Tiny / small threshold: 16 và 32
- IoU delta: tiny = 0.05, small = 0.02
- Top-k = 4, center radius scale = 1.3
- Min positive / tiny GT = 3

Bộ tỉ lệ cắt của ASR thay đổi theo thí nghiệm: bộ mặc định hoặc bộ 12 mức theo bài báo.

ASR và JSAR hoạt động như thế nào

ASR

- Bắt đầu từ một bộ ứng viên tỉ lệ cắt đủ rộng.
- Gom thống kê face-size distribution và thiếu hụt recall theo từng giai đoạn train.
- Điều chỉnh xác suất chọn các mức tỉ lệ cắt để tăng tần suất xuất hiện của nhóm tiny/small faces còn thiếu.
- Làm mượt phân phối xác suất bằng EMA để tránh dao động quá mạnh.

JSAR

- Tiny faces thường nhận quá ít mẫu dương với quy tắc gán gốc.
- JSAR nói lỏng bước chọn ứng viên quanh tâm.
- Nếu vẫn thiếu mẫu dương, fallback sẽ giữ lại một nhóm anchor gần nhất còn hợp lý.
- Nhờ đó tiny faces không còn bị rơi khỏi supervision.

ASR: công thức điều chỉnh phân phối tỉ lệ cắt

Đo độ khó theo từng nhóm kích thước

$$r_b = \max\left(0, 1 - \min\left(\frac{P_b}{\max(G_b, 1)}, 1\right)\right)$$

$$c_b = \frac{L_b^{cls}}{\max(P_b, 1)}, \quad u_b = \frac{L_b^{box}}{\max(P_b, 1)}$$

$$d_b \propto \text{norm}(r_b) + \text{norm}(c_b) + \text{norm}(u_b)$$

Cập nhật xác suất chọn tỉ lệ cắt

$$q(s) = \sum_b M(s, b) d_b$$

$$\hat{p}(s) = \frac{q(s)}{\sum_j q(j)}$$

$$p_{t+1}(s) = \alpha p_t(s) + (1 - \alpha) \text{floor}(\hat{p}(s))$$

- b : nhóm kích thước mặt; s : tỉ lệ cắt ảnh; $M(s, b)$: mức hỗ trợ giữa mỗi mức cắt và từng nhóm kích thước.
- ASR tăng xác suất cho các mức tỉ lệ cắt tạo ra nhiều tín hiệu học hơn ở các nhóm còn khó.
- Tỉ lệ cắt nhỏ tương ứng **zoom-in**; tỉ lệ cắt lớn tương ứng **zoom-out**.

JSAR: công thức nối phép gán mẫu cho tiny faces

Hạ ngưỡng và nối center gating

$$\tau'_g = \tau_g - \Delta_g, \quad \Delta_g = \begin{cases} 0.05, & \text{size}_g < 16 \\ 0.02, & 16 \leq \text{size}_g < 32 \\ 0, & \text{otherwise} \end{cases}$$

$$c_g = \text{clip}\left(\frac{\text{size}_g}{T_{\text{tiny}}}, 1, \rho\right)$$

keep if $\text{dist}_{\min}(a, g) c_g > t_{\text{center}}$

Fallback top-k nếu vẫn thiếu mẫu dương

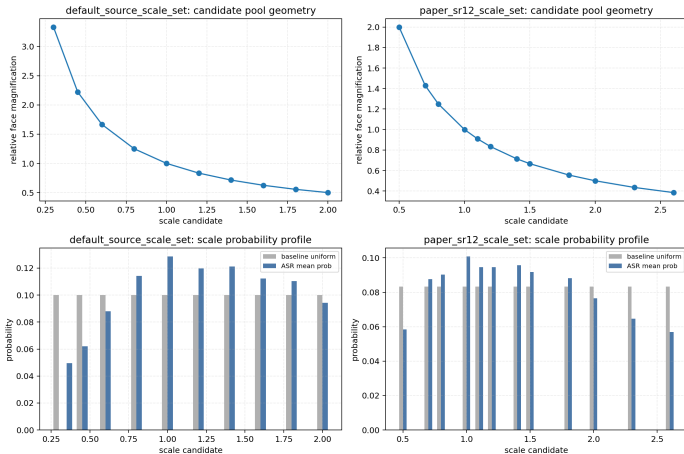
$$\text{score}(a, g) = \text{IoU}(a, g) - 0.05 \frac{\text{dist}(a, g)}{\max(\text{size}_g, 1)}$$

$$\mathcal{P}_g \leftarrow \mathcal{P}_g \cup \text{TopK}(\text{score}, k)$$

$$|\mathcal{P}_g| \geq N_{\min}, \quad N_{\min} = 3 \text{ cho tiny GT}$$

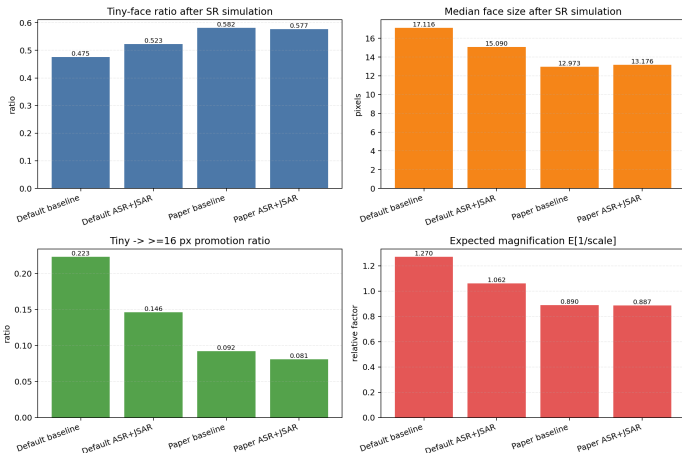
- JSAR chỉ can thiệp mạnh vào tiny/small faces; medium và large gần như giữ nguyên.
- Ý nghĩa chính: tiny faces không còn bị rơi khỏi supervision chỉ vì IoU thấp hoặc center constraint quá chặt.

ASR: hình học của bộ tỉ lệ cắt



- Tỉ lệ cắt nhỏ làm khuôn mặt lớn hơn sau resize; tỉ lệ cắt lớn làm khuôn mặt nhỏ hơn sau resize.

ASR: áp lực phân phối theo nhóm kích thước



- Khi một nhóm kích thước còn khó, ASR sẽ tăng xác suất cho các mức tỉ lệ cắt có khả năng tạo thêm tín hiệu học cho đúng nhóm đó.

Protocol thực nghiệm cho ASR+JSAR

- Cả baseline và cải tiến đều dùng **SCRFD-2.5GF**, input 640×640 , 80 epoch, cosine scheduler.
- Nhóm chạy hai cặp thí nghiệm để tách ảnh hưởng của bộ tỉ lệ cắt và cơ chế điều chỉnh động trong lúc train:

Hai trường hợp

Bộ mặc định trong mã nguồn: baseline gốc của source code.

Bộ 12 mức theo bài báo: tập tỉ lệ cắt gần hơn với narrative của paper.

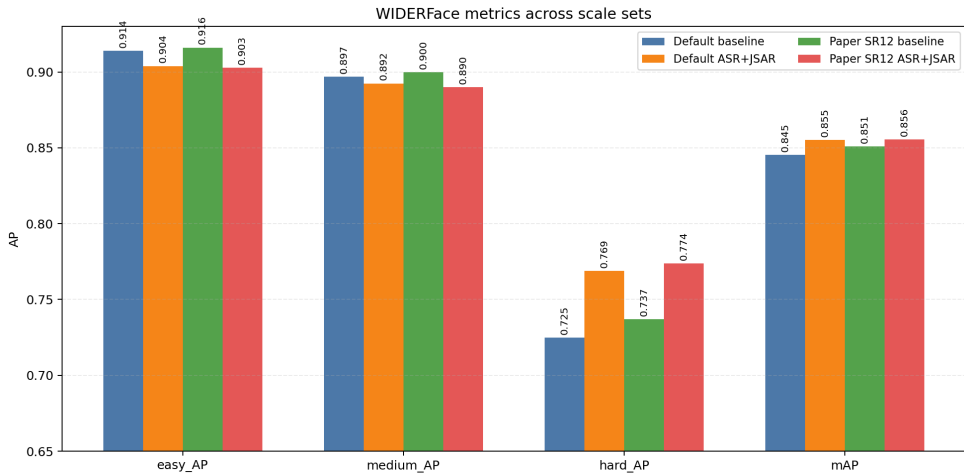
- Cách so sánh này giúp đọc đúng delta của ASR+JSAR: gain đến từ việc điều chỉnh động xác suất chọn tỉ lệ cắt và cách gán mẫu dương, không chỉ từ việc đổi bộ tỉ lệ cắt.
- Trên nhánh **mặc định**, baseline và ASR không dùng hoàn toàn cùng điểm đầu bên trái của bộ ứng viên; trên nhánh **12 mức theo bài báo**, cả hai mới thật sự dùng cùng một tập 12 mức.

Kết quả ASR+JSAR trên WIDER FACE

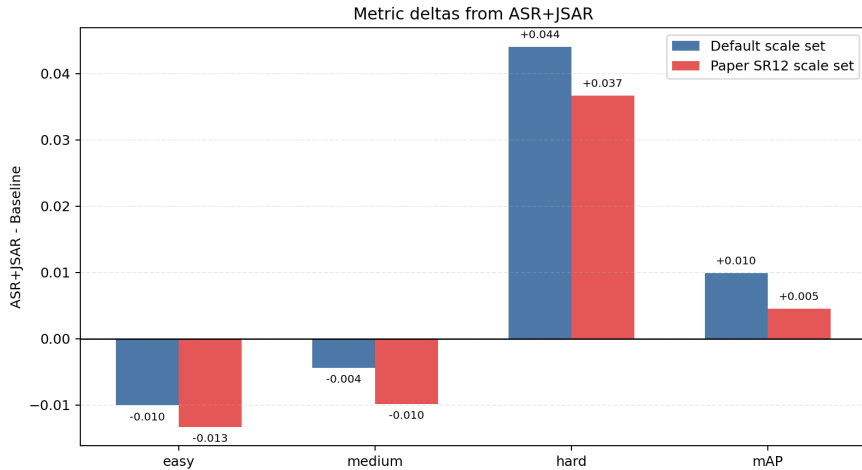
Thiết lập	Easy	Medium	Hard	mAP
Baseline (mặc định)	91.40	89.69	72.49	84.53
ASR+JSAR (mặc định)	90.39	89.25	76.90	85.51
Baseline (12 mức theo bài báo)	91.61	89.99	73.71	85.10
ASR+JSAR (12 mức theo bài báo)	90.28	89.01	77.38	85.56

- Gain lớn nhất tập trung ở **Hard**: +4.40 với bộ mặc định và +3.67 với bộ 12 mức theo bài báo.
- Trên bộ 12 mức theo bài báo, kết quả chỉ còn cách official model zoo khoảng 0.49 điểm ở split khó nhất.

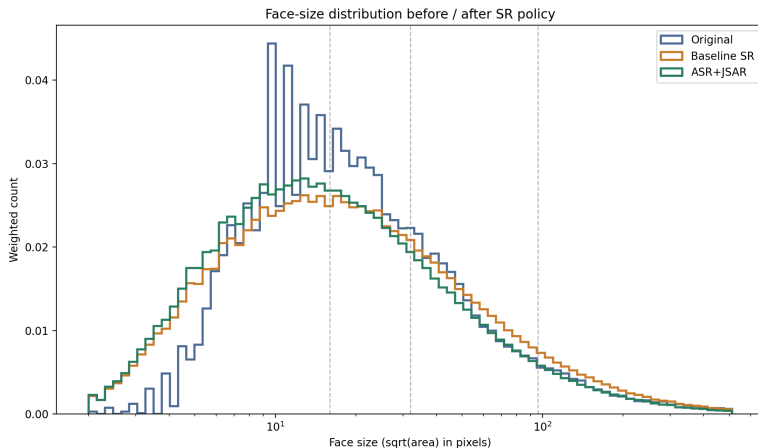
Các chỉ số theo từng bộ tỉ lệ cắt



Delta của ASR+JSAR theo từng bộ tỉ lệ cắt

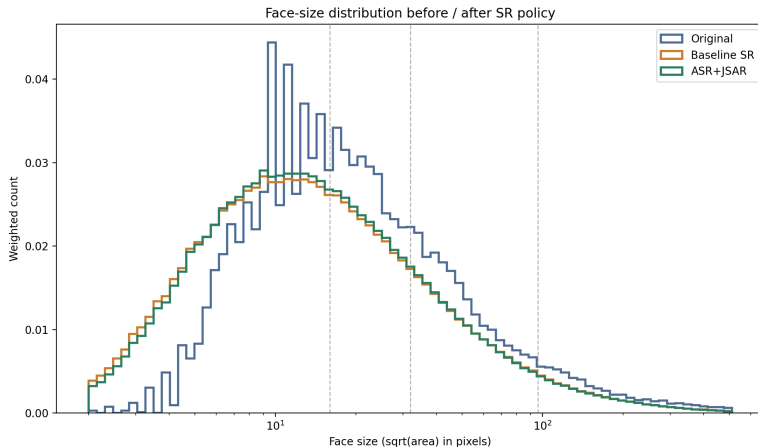


Histogram kích thước khuôn mặt: bộ mặt định trong mã nguồn



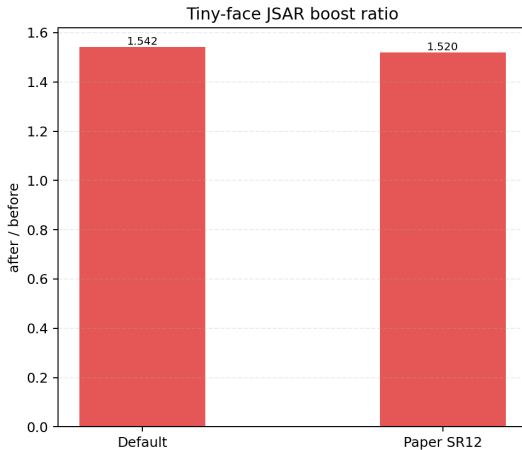
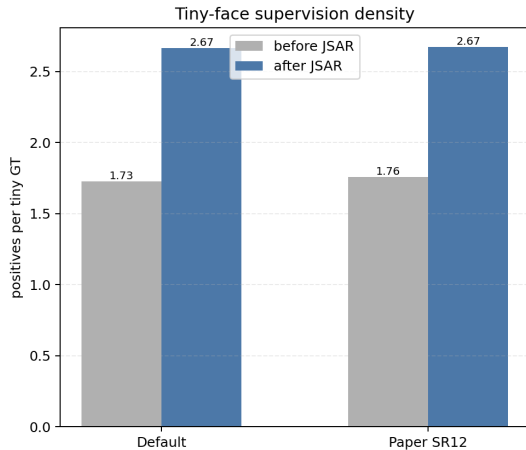
- Với bộ mặt định, histogram của ASR+JSAR lệch trái rõ hơn baseline, cho thấy mô hình cải tiến chủ động tạo thêm nhiều khuôn mặt nhỏ trong lúc huấn luyện.

Histogram kích thước khuôn mặt: bộ 12 mức theo bài báo

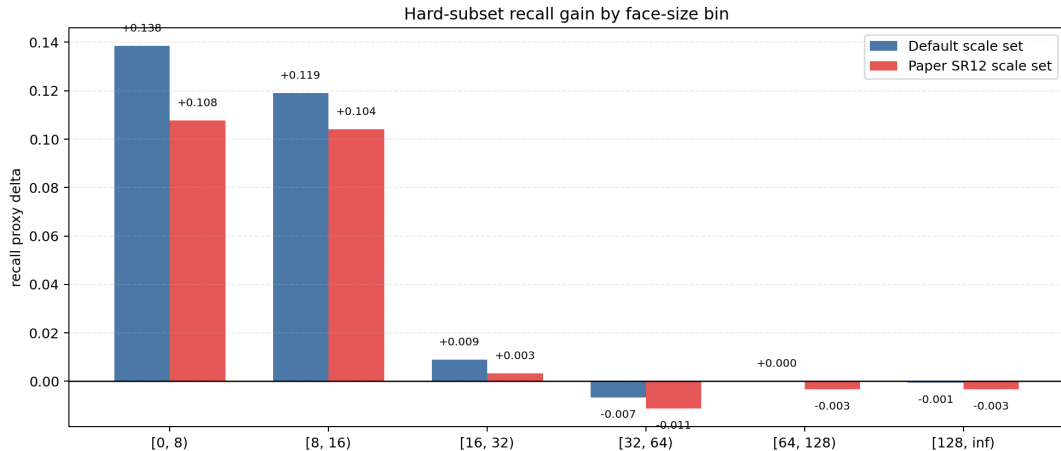


- Với bộ 12 mức theo bài báo, baseline và ASR+JSAR có phân phối gần nhau hơn; phần gain còn lại đến nhiều hơn từ cơ chế gán mẫu của JSAR.

JSAR: mật độ mẫu dương cho tiny faces



Gain theo nhóm khuôn mặt khó



- Kết quả hiện tại mới được đánh giá trong thiết lập huấn luyện 80 epoch, trong khi bản SCRFD gốc trong bài báo được huấn luyện dài hơn, tới 640 epoch. Vì vậy, chưa thể kết luận đây là mức trần hiệu quả của ASR+JSAR.
- Lợi ích của phương pháp tập trung rõ nhất ở khuôn mặt nhỏ và các mẫu thuộc split Hard; đây cũng là nơi SCRFD baseline yếu nhất.
- Đổi lại, khi tín hiệu học được dồn nhiều hơn về nhóm khó, hiệu năng trên khuôn mặt vừa và lớn có thể giảm nhẹ. Nói cách khác, cải tiến này không làm tốt đồng đều mọi nhóm kích thước, mà ưu tiên mạnh hơn cho nhóm khó nhất.

Kết luận

- SCRFD mạnh vì tối ưu đồng thời **nơi mô hình đặt năng lực biểu diễn** và **nơi dữ liệu tạo tín hiệu học**.
- **Baseline reproduction** cho thấy gap lớn nhất luôn nằm ở Hard; đó mới là vùng khó thật của SCRFD.
- **ASR+JSAR** là bằng chứng mạnh nhất của report: giữ nguyên kiến trúc suy luận vẫn có thể kéo Hard AP từ 73.71 lên 77.38 nếu phân phối tỉ lệ cắt và mật độ mẫu dương cho tiny faces được sửa đúng chỗ.
- **SPOS-NAS** củng cố phía CR: trong cùng miền 2.5 GFLOPs, winner thắng vì phân bổ compute đúng hơn, đặc biệt ở C_2 , C_3 và C_5 .
- Insight cuối cùng của report là: với efficient face detection, tối ưu kiến trúc thôi là chưa đủ; phải tối ưu cùng lúc **architecture, data distribution và supervision**.

Tài liệu tham khảo

Tài liệu tham khảo I



Jia Guo, Jiankang Deng, Alexandros Lattas, and Stefanos Zafeiriou.
Sample and Computation Redistribution for Efficient Face Detection.
In International Conference on Learning Representations, 2022.



Shuo Yang, Ping Luo, Chen Change Loy, and Xiaoou Tang.
WIDER FACE: A Face Detection Benchmark.
In IEEE Conference on Computer Vision and Pattern Recognition, 2016.



Jiankang Deng, Jia Guo, Yuxiang Zhou, Jinke Yu, Irene Kotsia, and Stefanos Zafeiriou.
RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild.
arXiv preprint arXiv:1905.00641, 2020.



Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár.
Focal Loss for Dense Object Detection.
In IEEE International Conference on Computer Vision, 2017.

Tài liệu tham khảo II



Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia.

Path Aggregation Network for Instance Segmentation.

In IEEE Conference on Computer Vision and Pattern Recognition, 2018.



Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi.

You Only Look Once: Unified, Real-Time Object Detection.

In IEEE Conference on Computer Vision and Pattern Recognition, 2016.



Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg.

SSD: Single Shot MultiBox Detector.

arXiv preprint arXiv:1512.02325, 2016.



Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun.

Deep Residual Learning for Image Recognition.

In IEEE Conference on Computer Vision and Pattern Recognition, 2016.



Shifeng Zhang, Cheng Chi, Yongqiang Yao, Zhen Lei, and Stan Z. Li.

Bridging the Gap Between Anchor-based and Anchor-free Detection via Adaptive Training Sample Selection.

In IEEE Conference on Computer Vision and Pattern Recognition, 2020.



DeepInsight contributors.

InsightFace SCRFD Repository and Documentation.

<https://github.com/deepinsight/insightface/tree/master/detection/scrfd>. Accessed 2026-04-19.

Cảm ơn thầy và các bạn đã lắng nghe

Trả lời câu hỏi